ICMC USP SÃO CARLOS

MAI5001

Introdução à Ciência de Computação

Busca Interna Avançada

- Busca sequencial indexada e busca por salto
- Busca por interpolação
- Busca exponencial
- Comparar, medir e escolher



ICMC USP SÃO CARLOS

MAI5001

Introdução à Ciência de Computação

Busca sequencial indexada e busca por salto

Organizar os dados em blocos para evitar percorrer todos os registros.



Problema motivador: acessos têm custos diferentes

Imagine 10.000 registros ordenados, armazenados em 100 blocos de 100 registros.

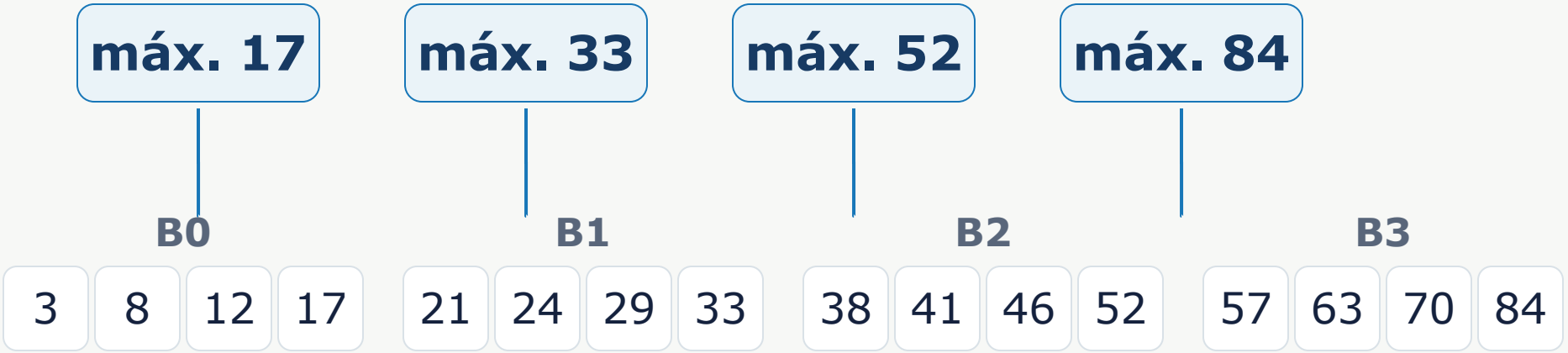
bloco 0	bloco 1	bloco 2	bloco 3	bloco 4	bloco 5	bloco 6	bloco 7
0 … 99	100 … 199	200 … 299	300 … 399	400 … 499	500 … 599	600 … 699	700 … 799

Pergunta

Como descobrir rapidamente qual bloco pode conter a chave procurada?

Busca sequencial indexada: duas camadas

Índice esparso



Leitura do índice

Cada entrada resume um bloco. Aqui, o valor guardado é a maior chave daquele bloco e aponta para sua localização.

O índice é menor que o conjunto de dados: há 1 entrada para cada bloco, não 1 entrada para cada registro.

Exemplo: localizar a chave 46

1 Percorrer o índice

17 < 46, 33 < 46 e 52 ≥ 46. Portanto, o primeiro bloco possível é B2.

2 Abrir B2

O bloco contém [38, 41, 46, 52]. Blocos anteriores já foram descartados.

3 Busca sequencial

38 < 46, 41 < 46 e 46 = 46. A chave foi encontrada.

Índice	17	33	**52**	84
Bloco B2	38	41	**46**	52

Resultado

Encontrado em B2, posição local 2 — posição global 10.

Algoritmo da busca sequencial indexada

```
buscaIndexada(indice, blocos, x):
  i ← 0
  enquanto i < tamanho(indice) e indice[i].max < x:
      i ← i + 1

  se i = tamanho(indice):
      retorne NÃO_ENCONTRADO

  para cada posição j do bloco apontado por indice[i]:
      se blocos[i][j] = x: retorne (i, j)
      se blocos[i][j] > x: interrompa

  retorne NÃO_ENCONTRADO
```

Pré-condição

Os blocos e as entradas do índice estão em ordem crescente.

Parada antecipada

Dentro do bloco, se o valor atual ultrapassa x, x não aparecerá depois.

Saída

Pode ser a posição global, um par (bloco, posição) ou o registro associado.

Por que o algoritmo está correto?

A prova informal usa duas exclusões seguras:

1 · Exclusão de blocos

Quando indice[k].max < x, todas as chaves do bloco k são menores que x. Logo, x não está ali.

2 · Exclusão dentro do bloco

Ao encontrar uma chave maior que x em um bloco ordenado, todas as posições seguintes também são maiores.

O único bloco que ainda pode conter x é pesquisado completamente (ou até ultrapassar x).

Custo quando o índice é percorrido sequencialmente

Se há n registros e cada bloco possui b registros:

n / b
entradas no índice

+

b
comparações no bloco

=

Θ(n/b + b)
pior caso

Escolha equilibrada

O custo é minimizado, em ordem de grandeza, quando b ≈ √n. Assim, as duas parcelas ficam próximas.

Exemplo

n = 10.000 e b = 100 ⇒ até cerca de 100 entradas no índice + 100 comparações no bloco.

Importante: se o custo dominante for acesso a página, conte páginas — não apenas comparações.

E se buscarmos o índice com busca binária?

A primeira camada também é uma sequência ordenada.

Índice sequencial

Θ(n/b) para escolher o bloco
+ Θ(b) dentro do bloco

Índice binário

Θ(log(n/b)) para escolher o bloco
+ Θ(b) dentro do bloco

Custo total

Θ(log(n/b) + b)

Quando o índice cabe em memória e os blocos estão em armazenamento mais lento, essa organização aproxima a análise do comportamento real do sistema.

NÃO CONFUNDIR

O índice acelera a localização do bloco; ele não elimina a necessidade de pesquisar o bloco escolhido.

Busca por salto: a mesma estratégia sem índice

Em vez de guardar uma tabela auxiliar, visitamos periodicamente o próprio vetor.

3	8	12	**17**	21	24	29	**33**	38	41	46	**52**	57	63	70	**84**
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
			salto 1				**salto 2**				**salto 3**				**salto 4**

1 Saltar

Compare posições separadas por b elementos.

2 Ultrapassar

Pare quando A[j] ≥ x ou o vetor terminar.

3 Voltar e varrer

Faça busca sequencial no bloco anterior.

Exemplo: saltos de tamanho 4 para encontrar 46

3	8	12	**17**	21	24	29	**33**	38	41	46	**52**	57	63	70	84
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Provas por salto

A[3]=17 < 46 → A[7]=33 < 46 → A[11]=52 ≥ 46

Intervalo restante

Pesquisar sequencialmente de 8 a 11: 38, 41, 46 → encontrado em 10.

Por que voltar para 8?

A posição 7 foi o último salto com valor menor que 46; logo, a chave só pode estar depois dela e até a posição 11.

Algoritmo e custo da busca por salto

```
buscaSalto(A, x, b):
  anterior ← 0
  atual ← b

  enquanto atual < n e A[atual - 1] < x:
      anterior ← atual
      atual ← atual + b

  limite ← min(atual, n)
  para i de anterior até limite - 1:
      se A[i] = x: retorne i
      se A[i] > x: interrompa

  retorne NÃO_ENCONTRADO
```

n / b
saltos

b
varredura

$$C(b) = \Theta(n/b + b)$$

$$b \approx \sqrt{n} \Rightarrow \Theta(\sqrt{n})$$

Espaço extra
$\Theta(1)$ na versão iterativa.

Comparação: qual estrutura está sendo explorada?

Método	Informação adicional	Pior caso	Quando considerar
Sequencial indexada	Índice por blocos	$\Theta(n/b + b)$	Dados em blocos; índice já existe
Busca por salto	Tamanho do salto b	$\Theta(\sqrt{n})$ com $b \approx \sqrt{n}$	Estratégia simples, sem índice persistente
Busca binária	Apenas ordem + acesso aleatório	$\Theta(\log n)$	Opção geral para vetor ordenado em memória

Síntese e referências

Leitura correta

A **busca binária** exige d**ados ordenados e acesso direto eficiente** a qualquer posição. Em vetores armazenados em memória, cada acesso por índice custa Θ(1), permitindo uma busca em Θ(log n). Em **estruturas sem acesso aleatório eficiente**, como listas encadeadas, a b**usca binária perde essa vantagem**. Além disso, o custo de ordenar os dados, caso ainda não estejam ordenados, não está incluído na complexidade da busca.

O que fica

Índice e saltos seguem a mesma estratégia: localizar uma região e fazer uma busca local. O tamanho do bloco controla o equilíbrio entre as duas etapas.

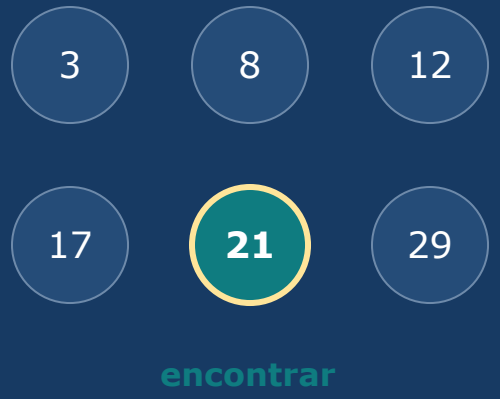
ICMC USP SÃO CARLOS

MAI5001

Introdução à Ciência de Computação

Busca por interpolação

Estimar a posição provável usando o valor da chave — e reconhecer quando essa estimativa falha.



Da divisão fixa à estimativa orientada pelos valores

Busca binária

Olha sempre para o meio do intervalo, independentemente do valor procurado.

Busca por interpolação

Estima onde x deveria aparecer entre A[início] e A[fim].

10	20	30	40	50	60	**70**	80	90

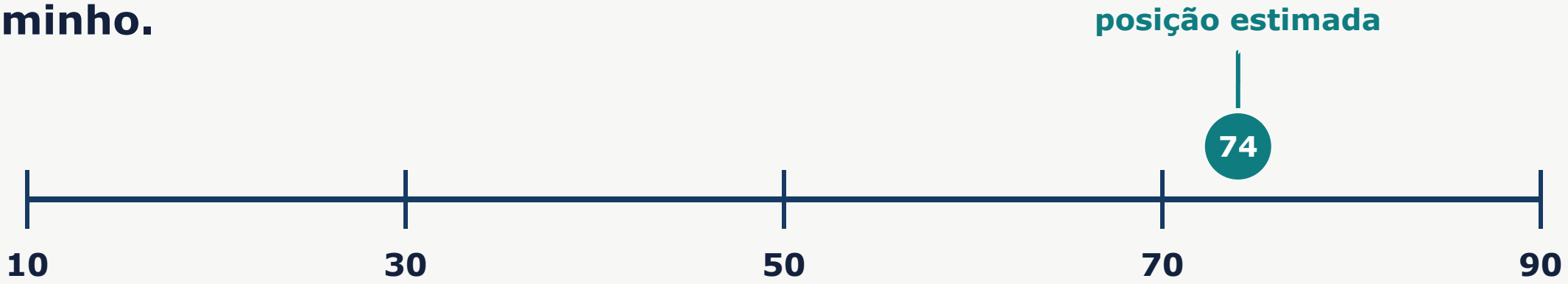
meio = 50

estimativa para x=70

A interpolação pode saltar para perto da resposta — mas só se os valores estiverem distribuídos de forma favorável.

Intuição: uma régua de valores

Se 10 está no início e 90 está no fim, 74 parece estar perto de 80% do caminho.



Analogia

É como procurar uma palavra em um dicionário: para “sistema”, abrimos mais perto do fim do que do começo.

A fórmula de interpolação

A analogia ajuda na intuição; a fórmula abaixo torna a estimativa precisa.

$$pos = início + \lfloor (x - A[início]) \cdot (fim - início) / (A[fim] - A[início]) \rfloor$$

1 · Fração do valor

(x − A[início]) / (A[fim] − A[início]) indica quão longe x está entre os valores extremos.

2 · Fração do intervalo

Multiplicamos pela largura (fim − início) para converter essa proporção em deslocamento de índices.

Usamos piso ⌊·⌋ para obter um índice inteiro e limitamos o cálculo ao intervalo válido.

Quando a estimativa é válida?

Obrigatório

Vetor ordenado em ordem crescente e acesso direto aos elementos.

Tipo de chave

A fórmula precisa de distância numérica, ou de uma transformação que preserve a ordem.

Cenário favorável

Chaves aproximadamente uniformes: diferenças de valor acompanham diferenças de posição.

Cuidado

Se A[início] = A[fim], o denominador é zero. O caso deve ser tratado antes da fórmula.

Valores repetidos não tornam o algoritmo impossível, mas exigem guardas e uma política clara para retornar qualquer ocorrência, a primeira ou a última.

Exemplo 1: a primeira estimativa acerta

10	18	26	34	42	50	58	66	**74**	82	90
0	1	2	3	4	5	6	7	8	9	10

início = 0, fim = 10, x = 74

pos = 0 + ⌊(74−10) · 10 / (90−10)⌋
= ⌊640 / 80⌋
= 8

Verificação

A[8] = 74. A chave foi encontrada com uma única estimativa.

Isso é excelente — mas não é garantia. O próximo exemplo mostra um ajuste de intervalo.

Exemplo 2: estimar, comparar e ajustar

10	12	15	19	24	30	37	45	**54**	**64**	75
0	1	2	3	4	5	6	7	8	9	10

Estimativa 1

pos = ⌊(64−10)·10/(75−10)⌋ = 8
A[8] = 54 < 64 ⇒ início ← 9

Estimativa 2

Com início = 9: pos = 9
A[9] = 64 ⇒ encontrado

1 Estimar

Calcular pos com os extremos atuais.

2 Comparar

Observar A[pos] em relação a x.

3 Reduzir

Descartar um lado e repetir.

Algoritmo da busca por interpolação

```
buscaInterpolacao(A, x):
  início ← 0; fim ← n − 1

  enquanto início ≤ fim e A[início] ≤ x ≤ A[fim]:
      se A[início] = A[fim]:
          se A[início] = x: retorne início
          senão: interrompa

      pos ← início + piso(
              (x − A[início]) · (fim − início)
              / (A[fim] − A[início]))

      se A[pos] = x: retorne pos
      se A[pos] < x: início ← pos + 1
      senão: fim ← pos − 1

  retorne NÃO_ENCONTRADO
```

Guarda do laço

x precisa estar entre os valores extremos do intervalo. Se sair dessa faixa, não pode estar no vetor ordenado.

Implementação

Use um tipo numérico largo no produto para evitar overflow antes da divisão.

Índice válido

A fórmula deve produzir início ≤ pos ≤ fim; verifique limites ao adaptar para tipos e linguagens diferentes.

Por que termina e por que encontra a chave?

Invariante informal

Se x está no vetor, então x está entre início e fim no começo de cada iteração.

1 A[pos] < x

Tudo até pos é pequeno demais; início recebe pos+1.

2 A[pos] = x

A posição procurada foi encontrada e o algoritmo termina.

3 A[pos] > x

Tudo desde pos é grande demais; fim recebe pos−1.

Progresso

Quando não encontra x, o intervalo perde pelo menos uma posição. Portanto, o laço não pode repetir para sempre.

Complexidade: excelente em média, frágil no pior caso

Θ(log log n)
caso médio sob chaves uniformes

Θ(n)
pior caso

Θ(1)
espaço extra iterativo

Hipótese do resultado médio

As chaves devem se comportar aproximadamente como amostras uniformemente distribuídas. A ordem, sozinha, não basta.

Sem essa hipótese, a estimativa pode avançar apenas uma posição por iteração.

Por que a redução por raiz quadrada é especial?

Se a cada iteração o intervalo restante passa de n para $\sqrt{n}$, o expoente é dividido por 2 a cada passo.

Tamanho do intervalo

Iteração 0: n

Iteração 1: $n^{(1/2)}$

Iteração 2: $n^{(1/4)}$

Iteração 3: $n^{(1/8)}$

Iteração k: $n^{(1/2k)}$

A cada passo, o problema não é apenas "diminuído"; ele é reduzido para uma potência com expoente cada vez menor.

Após 1 passo: $n \longrightarrow n^{(1/2)}$.

Após 2 passos: $n^{(1/2)} \longrightarrow (n^{(1/2)})^{(1/2)} = n^{(1/4)}$.

Após 3 passos: $n^{(1/4)} \longrightarrow (n^{(1/4)})^{(1/2)} = n^{(1/8)}$.

Em geral, após k passos: $n_k = n^{((1/2)^k)}$.

Ideia-chave: extrair raiz quadrada repetidamente equivale a dividir o expoente por 2 repetidas vezes.

Quantas iterações são necessárias?

Queremos descobrir o menor k tal que o intervalo restante tenha tamanho constante.

$n^{(½)^k} \leq 2$

Aplicando $\log_2$ nos dois lados:
$(1/2^k) \cdot \log_2 n \leq \log_2 2 = 1$

Logo, $\log_2 n \leq 2^k$

Aplicando $\log_2$ novamente:
$\log_2(\log_2 n) \leq k$

Conclusão

O número de iterações necessário é aproximadamente o menor k que satisfaz $\log_2(\log_2 n) \leq k$.

Portanto, $k = \lceil \log_2(\log_2 n) \rceil$.

Se cada iteração custa O(1), então o tempo total é O(log log n).

Exemplo: se n = 2^16 = 65.536, então $\log_2 n = 16$ e $\log_2 16 = 4$.

Resultado final: T(n) = O(log log n)

A mesma ideia via recorrência

Podemos obter o mesmo resultado modelando a busca por uma recorrência idealizada.

Recorrência idealizada

$$T(n) = T(\sqrt{n}) + O(1)$$

Cada passo custa $O(1)$ e reduz o problema de n para aproximadamente $\sqrt{n}$.

Faça a mudança de variável: $m = \log_2 n$ (logo, $n = 2^m$).

Então: $\sqrt{n} = \sqrt{(2^m)} = 2^{(m/2)}$.

Defina: $S(m) = T(2^m)$.

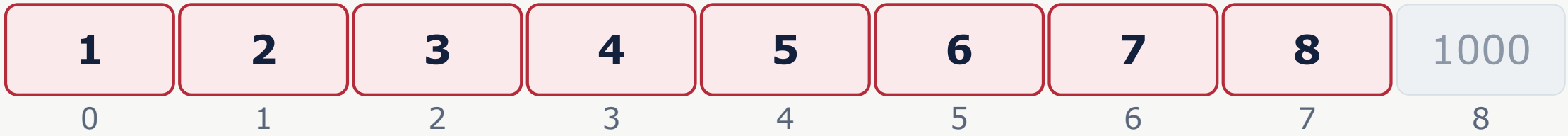
Substituindo na recorrência:
$S(m) = S(m/2) + O(1)$.

Como m é reduzido pela metade a cada passo, temos $S(m) = O(\log m)$.

Voltando para n: $T(n) = O(\log(\log n))$.

Interpretação: tirar a raiz de n corresponde a dividir log n por 2; por isso, a profundidade do processo é log log n.

Contraexemplo: distribuição extremamente assimétrica



Procurando x = 8

Como 1000 domina o intervalo de valores, a fórmula estima posições próximas do início: 0, depois 1, depois 2…

Consequência

A busca se aproxima de uma varredura sequencial e pode realizar Θ (n) comparações.

Moral: distribuição dos valores é parte da entrada do algoritmo.

Armadilhas de implementação

Situação	Risco	Tratamento
A[início] = A[fim]	Divisão por zero	Comparar diretamente com x
Produto muito grande	Overflow inteiro	Promover para tipo mais largo
x fora dos extremos	Índice inválido	Manter a guarda A[início] ≤ x ≤ A[fim]
Valores repetidos	Qual ocorrência retornar?	Definir contrato: qualquer, primeira ou última
Ponto flutuante	Arredondamento	Testar limites e evitar igualdade ingênua quando cabível

Regra prática

A prova do algoritmo pressupõe que pos permanece no intervalo. Guardas de implementação preservam essa condição no programa real.

Busca binária ou interpolação?

Critério	Busca binária	Busca por interpolação
Usa os valores?	Não; usa apenas a ordem	Sim; usa distância entre valores
Pior caso	Θ(log n)	Θ(n)
Caso médio favorável	Θ(log n)	Θ(log log n) sob uniformidade
Tipos de chave	Qualquer tipo ordenável	Chaves numéricas/mapeáveis
Robustez	Alta	Depende da distribuição

Padrão seguro: use busca binária. Considere interpolação quando a distribuição for conhecida ou validada por medição.

Síntese e referências

O que fica

A interpolação estima a posição com os valores extremos. O ganho esperado depende da distribuição; a correção depende das guardas e da redução segura do intervalo.

ICMC USP SÃO CARLOS

MAI5001

Introdução à Ciência de Computação

Busca exponencial

Descobrir rapidamente um intervalo de busca quando o limite superior não é conhecido.



Motivação: e se não conhecermos o fim?

Imagine uma sequência ordenada em que podemos consultar qualquer posição valor(i), mas não sabemos quantos elementos existem. Como determinar o intervalo inicial necessário para realizar uma busca binária?

A busca binária precisa de um intervalo [início, fim].

Vetor virtual

Uma API responde valor(i), mas não informa antecipadamente o tamanho total.

Coleção muito grande

O alvo pode estar perto do início; começar com todo o intervalo desperdiça informação de posição.



Testar 1, 2, 4, 8... dobra o alcance a cada passo.

DUAS FASES **1) cercar a resposta; 2) procurar dentro do intervalo.**

Fase 1: expandir até cercar a chave

Começamos no índice 1 e dobramos enquanto A[i] < x.



índices consultados

Parada
Quando A[i] ≥ x, ou quando i ultrapassa o último índice do vetor.

Intervalo descoberto
A chave, se existir, está entre i/2 e i respeitando os limites do vetor.

Exemplo: encontrar 53

2	**5**	**8**	12	**17**	21	26	30	**35**	39	44	48	50	53	59	63	**68**	72	77	81
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Provas da fase exponencial

A[1]=5 < 53 → A[2]=8 < 53 → A[4]=17 < 53 → A[8]=35 < 53 → A[16]=68 ≥ 53

Conclusão

A chave, se existir, está no intervalo de índices [8, 16].

A fase 1 não encontra necessariamente x; ela encontra um limite superior suficiente.

Fase 2: busca binária no intervalo [8, 16]

Índices globais
8 .. 16

35	39	44	48	**50**	**53**	**59**	63	68

1 meio = 12

A[12]=50 < 53;
novo início = 13.

2 meio = 14

A[14]=59 > 53;
novo fim = 13.

3 meio = 13

A[13]=53;
encontrado.

Expansão escolhe o intervalo; busca binária escolhe a posição.

O algoritmo completo em uma frase

1 Caso base

Se A[0] = x, termine imediatamente.

2 Dobrar

Encontre i com $A[i/2] < x \leq A[i]$, ou atinja n.

3 Dividir

Execute busca binária entre i/2 e min(i,n−1).

[1, 2, 4, 8, ...] → [i/2, i] → busca binária

Algoritmo para vetor finito

```
buscaExponencial(A, x):
  n ← tamanho(A)
  se n = 0: retorne NÃO_ENCONTRADO
  se A[0] = x: retorne 0

  i ← 1
  enquanto i < n e A[i] < x:
      i ← 2 · i

  início ← i / 2
  fim ← min(i, n − 1)
  retorne buscaBinaria(A, x, início, fim)
```

Limite seguro

min(i, n−1) impede acesso fora do vetor quando o último salto ultrapassa n.

Pré-condição

A deve estar ordenado. A busca binária recebe apenas o intervalo descoberto.

A implementação pode reutilizar diretamente a função de busca binária.

E em uma sequência de tamanho desconhecido?

Substituímos o teste i < n por uma operação segura de leitura.

```
i ← 1
enquanto valor(i) existe e valor(i) < x:
    i ← 2 · i

// agora: fim da sequência foi atingido
// ou valor(i) ≥ x
```

Contrato da API

valor(i) precisa sinalizar claramente "posição inexistente" sem comprometer a ordem dos valores válidos.

Depois

A busca binária também precisa saber como tratar posições inexistentes — frequentemente como +∞ para fins de comparação.

O detalhe operacional muda; a ideia matemática de dobrar o limite permanece.

Por que a chave permanece no intervalo?

Antes do último salto

A[i/2] < x. Portanto, x não pode estar em posição ≤ i/2 se as chaves são crescentes (salvo duplicatas, conforme o contrato).

No último salto

A[i] ≥ x, ou i excedeu o fim. Assim, não precisamos procurar depois do limite superior válido.

Logo, se x existe, uma ocorrência está em [i/2, min(i,n−1)].

Complexidade em função da posição p

Após k dobramentos, o índice alcançado é 2^k.

$k = \lceil \log_2 p \rceil$

passos de expansão

$\Theta(\log p)$

busca binária no intervalo

$\Theta(\log p)$

custo total

Pior caso finito

Se x está no fim ou não existe e é maior que todos os valores, p se comporta como n: Θ(log n).

Espaço extra: Θ(1) com busca binária iterativa; Θ(log p) se a fase binária for recursiva.

Quando o alvo está perto do início

Em uma coleção enorme, p pode ser muito menor que n.

Exemplo ilustrativo

n = 1.000.000.000 e a chave está na posição p = 8.

Busca binária global

Cerca de $\log_2 n \approx 30$ níveis para o intervalo completo.

Busca exponencial: prova 1, 2, 4, 8 e faz busca binária somente em [4,8].

Interpretação

O ganho vem de adaptar o trabalho à posição real, não apenas ao tamanho total.

Casos de fronteira

Entrada	O que acontece	Tratamento
Vetor vazio	Não há A[0]	Retornar não encontrado antes de acessar
x = A[0]	Não precisa expandir	Caso base em $\Theta(1)$
x < A[0]	Nenhum índice serve	Retornar não encontrado
x maior que todos	i ultrapassa n	Usar fim = min(i,n−1)
Chaves repetidas	Pode achar qualquer ocorrência	Ajustar se o contrato exigir a primeira

Teste mínimo

Inclua casos para n = 0, n = 1, chave ausente abaixo do mínimo e chave ausente acima do máximo.

Comparação de estratégias

Método	Conhece n?	Usa distribuição?	Pior caso	Ponto forte
Binária	Sim	Não	Θ(log n)	Robustez geral
Interpolação	Sim	Sim	Θ(n)	Chaves quase uniformes
Exponencial	Não necessariamente	Não	Θ(log n)	Custo Θ(log p)
Salto	Sim	Não	Θ(√n)	Simplicidade sem índice

Exponencial não substitui a busca binária: ela a prepara quando o limite superior ainda não é conhecido.

Síntese e referências

O que fica

A busca exponencial cria um intervalo por dobramentos e então aplica busca binária. Seu custo acompanha a posição p, o que é útil quando o limite é desconhecido ou o alvo está perto do início.

ICMC USP SÃO CARLOS

MAI5001

Introdução à Ciência de Computação

Comparar, medir e escolher

Transformar hipóteses de complexidade em decisões justificadas de projeto.



Antes de escolher: quais hipóteses são verdadeiras?

1. Os dados estão ordenados?
2. Há acesso aleatório barato?
3. As chaves são numéricas e quase uniformes?
4. O tamanho ou limite superior é conhecido?
5. Os dados são acessados em blocos?

Se uma hipótese muda, o algoritmo recomendado também pode mudar.

O que medir?

comparações
operações abstratas

acessos
leituras de memória/bloco

tempo
execução observada

memória
espaço auxiliar

Não são a mesma coisa

Duas implementações podem fazer o mesmo número de comparações e ter tempos diferentes por cache, linguagem, compilador ou layout dos dados.

A análise assintótica explica crescimento; a medição explica uma plataforma e uma carga de trabalho específicas.

Resumo teórico dos custos

Método	Pré-condição principal	Custo típico/favorável	Pior caso	Espaço
Sequencial	Nenhuma ordem	Θ(n)	Θ(n)	Θ(1)
Binária	Ordenado + acesso aleatório	Θ(log n)	Θ(log n)	Θ(1) iter.
Seq. indexada	Ordenado + índice/blocos	Depende do índice	Θ(n/b+b)*	Índice
Salto	Ordenado	Θ(√n) com b≈√n	Θ(√n)	Θ(1)
Interpolação	Ordenado + valores quase uniformes	Θ(log log n)**	Θ(n)	Θ(1)
Exponencial	Ordenado	Θ(log p)	Θ(log n)	Θ(1) iter.

* índice percorrido sequencialmente | ** sob hipótese de distribuição aproximadamente uniforme

Síntese

Pergunta sobre o cenário	Estratégia que ela sugere
Há um índice por blocos?	Busca sequencial indexada
A posição acompanha o valor?	Busca por interpolação, após validar
O limite superior é desconhecido?	Busca exponencial + binária
Nada especial é conhecido além da ordem?	Busca binária como padrão

Competência central

Ler as hipóteses, escolher o modelo de custo e justificar o algoritmo — não apenas memorizar pseudocódigo.

Referências da unidade

KNUTH, D. E. The Art of Computer Programming, v. 3: Sorting and Searching. 2. ed. Addison-Wesley, 1998.

SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley, 2011. Material complementar: Algorithms, 4th Edition.

PERL, Y.; ITAI, A.; AVNI, H. Interpolation Search—A Log LogN Search. Communications of the ACM, 21(7), 550–553, 1978. DOI: 10.1145/359545.359557.

BENTLEY, J. L.; YAO, A. C.-C. An Almost Optimal Algorithm for Unbounded Searching. Information Processing Letters, 5(3), 82–87, 1976. DOI: 10.1016/0020-0190(76)90071-5.

CORMEN, T. H. et al. Algoritmos: teoria e prática. Referência indicada na bibliografia da disciplina.